

Boucles et listes

TME 3

Objectifs

- ★ Structure conditionnelle
- ★ Boucle
- ★ Tests

Rendu

- ⇒ Les réponses seront écrites dans un fichier `nom.n°etu.tp3` où `nom` est votre nom et `n°etu` est votre numéro étudiant.
- ⇒ Si vous avez plusieurs fichiers :
 - Créer un répertoire `rendu TP3_votrenom.n°etu` où `votrenom` est votre nom et `n°etu` est votre numéro étudiant.
 - Copier l'ensemble des fichiers à rendre dans ce répertoire.
 - Archivez le répertoire à l'aide de la commande suivante

```
tar czvf tp3_votrenom.n°etu.tgz TP3_votrenom.n°etu
```
- ⇒ Charger votre fichier sur Moodle dans la devoir TP3

1 Problèmes

Exercice 1 Codage RLE

Le codage (run-length-encoding) est un algorithme de compression de données sans pertes. L'idée est de compresser les séquences de mêmes valeurs. Par exemple, le mot "aaaabbbccddd" sera codé "4a3b1c3d".

Question 1

Écrire une fonction `decode_rle` qui lit une chaîne de caractères qui contient une suite composée d'un entier et d'un caractère et retourne le mot décodé.

Question 2

Écrire une fonction `encode_rle` qui lit une chaîne de caractères et retourne le mot codé.

Exercice 2 Jeu du pendu

Le jeu du pendu consiste à trouver un mot en ne dépassant pas un nombre maximum de tentatives. A chaque tour le joueur peut demander une lettre, si cette lettre est contenue dans le mot alors on affiche l'endroit (ou les endroits) où cette lettre se trouve sinon le joueur a perdu une tentative. Le jeu s'arrête quand le mot est trouvé ou quand le joueur est pendu (a atteint le nombre de coup max) !

Question 1

Écrire une fonction `demande_lettre`, qui demande à l'utilisateur une lettre tant que le caractère n'est pas une lettre (Vous pouvez vous référer à la méthode `isalpha` des chaînes de caractères). La fonction retourne la lettre en minuscule (méthode `lower()`).

```
>>> demande_lettre()
Entrez une lettre : 3
Entrez une lettre : p
'p'
>>> demande_lettre()
Entrez une lettre : M
'm'
```

Question 2

Écrire une fonction `indice_lettre` qui prend en paramètre une chaîne et une lettre et retourne la liste des indices où se trouvent les lettres.

```
>>> indices_lettre("lalal", 'a')
[1, 3]
>>> indices_lettre("baobab", 'b')
[0, 3, 5]
>>> indices_lettre("baobab", 'x')
[]
```

Question 3

Écrire une fonction `decouvre` qui retourne le mot mystère avec des tirets à la place des lettres saufs si les indices sont dans une liste passer en paramètre.

```
>>> decouvre("lalal", [])
'-----'
>>> decouvre("baobab", [0, 2, 3, 5])
'b-ob-b'
>>> decouvre("lalal", [1, 3])
'-a-a-'
```

Question 4

Écrire le jeu du pendu qui demande à un joueur un mot secret et demande à un autre de le découvrir en un nombre de coup limité à 10.

2 Exercices à rendre

Pour ces exercices vous pouvez récupérer les données d'entrée sur moodle. Le code est à remettre dans le test **Challenge - 1** disponible sur moodle.

Exercice 3 la concentration de poussière

Les fichiers `releveX.txt` contiennent les valeurs de la concentration de *Dust* dans Oxford. Nous voulons savoir combien de fois cette concentration à augmenter. Par exemple, avec un relevé comme suit :

```
196
201
207
212
200
208
235
256
254
263
```

On regarde les changements comme ceci :

```
196 (rien - pas de valeur précédente)
201 (augmentation)
207 (augmentation)
212 (augmentation)
200 (diminution)
208 (augmentation)
235 (augmentation)
256 (augmentation)
254 (diminution)
263 (augmentation)
```

Écrire une fonction qui compte le nombre d'augmentation de *Dust* et vérifier que le nombre d'augmentation pour le `releve1.txt` est de 1705, et déterminer les résultats pour les autres relevés.

Exercice 4 Nouvelle mesure

En fait le doyen, n'est pas convaincu par vos résultats, il souhaiterait plutôt connaître le nombre de fois où il y a une augmentation sur une fenêtre de 3 mesures.

En reprenant l'exemple précédent :

```
196 A
202 A B
207 A B C
211 B C D
200 E C D
207 E F D
235 E F G
256 F G H
254 G H
263 H
```

Si on compare les 2 premières fenêtres de mesures :

- La première fenêtre marquée A (196,201,207) a une somme égale à $196 + 201 + 207 = 604$.
- La seconde fenêtre marquée B (201,207,212) a une somme égale à $201 + 207 + 212 = 620$
- La somme des concentrations de la fenêtres B est plus grande que la somme de la fenêtre A, on a donc une augmentation.

Votre objectif est de maintenant compter le nombres de fois où la somme des fenêtres glissantes augmente.

Pour notre exemple précédent :

```
A: 605 (rien - pas de valeur précédente)
B: 620 (augmentation)
C: 618 (diminution)
D: 618 (pas de changement)
E: 642 (augmentation)
F: 698 (augmentation)
G: 745 (augmentation)
H: 773 (augmentation)
```

Écrire une fonction qui compte le nombre d'augmentation de *Dust* et vérifier que le nouveau nombre d'augmentation pour le `relevel.txt` est de 1739, et déterminer les résultats pour les autres relevés.